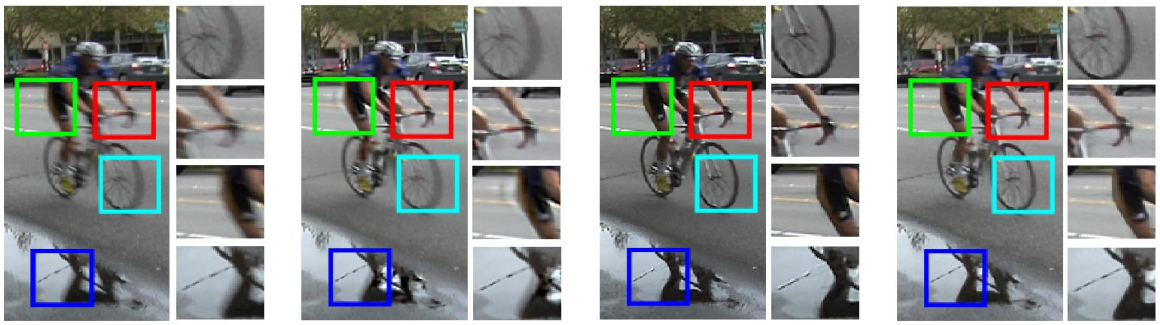
arXiv:1701.01486v1 [cs.CV] 5 Jan 2017

Motion Deblurring in the Wild

Mehdi Noroozi
Institute of Informatics
University of Bern
noroozi@inf.unibe.ch

Paramanand Chandramouli
Institute of Informatics
University of Bern
chandra@inf.unibe.ch

Paolo Favaro
Institute of Informatics
University of Bern
paolo.favaro@inf.unibe.ch



(a) (b) (c) (d)

Figure 1: Direct deblurring example. (a) Blurry video frame. (b) Result of [36] on the *single* frame in (a). (c) Output of video deblurring method by [17]. (d) Result of the proposed method using a *single* frame in (a) is comparable to video deblurring which uses multiple frames.

Abstract

The task of image deblurring is a very ill-posed problem as both the image and the blur are unknown. Moreover, when pictures are taken in the wild, this task becomes even more challenging due to the blur varying spatially and the occlusions between the object. Due to the complexity of the general image model we propose a novel convolutional network architecture which directly generates the sharp image. This network is built in three stages, and exploits the benefits of pyramid schemes often used in blind deconvolution. One of the main difficulties in training such a network is to design a suitable dataset. While useful data can be obtained by synthetically blurring a collection of images, more realistic data must be collected in the wild. To obtain such data we use a high frame rate video camera and keep one frame as the sharp image and frame average as the corresponding blurred image. We show that this realistic dataset is key in achieving state-of-the-art performance and dealing with occlusions.

1. Introduction

Motion blur artifacts are a major concern while capturing pictures in real world scenarios. Because of camera and object motion and due to the finite exposure interval, the image of different scene points gets averaged on the sensor causing motion blur (see Fig. 1 left). Motion deblurring has been exhaustively studied for the case when blur is uniform across the image. In this scenario, blind deconvolution algorithms achieve very good performance [37, 28]. However, in practice, motion blur varies across an image due to object motion, camera shake and the 3D structure of the scene. Modeling the relationship between the blurred observation and the sharp latent image by considering all these factors, in addition to occlusion effects, is quite a complicated task. Consequently, algorithms that handle space-variant blur either address only specific scenarios [15, 30, 40] or simplify the blur model [21, 36].

The common approach followed by deblurring algorithms is to initially determine the point spread function (PSF) that describes the blur and then solve for the sharp image [27, 26, 28]. Since the space-variant PSF is directly linked with the unknown factors such as scene depth, camera motion and segmentation boundaries of objects, deter-

mining it in a general setting is quite challenging. In this paper, we follow a different approach. Rather than enforcing a model to relate the blurred observation and the sharp image, we attempt to directly recover the latent sharp image by discriminative learning through a deep network formulation. We learn the parameters of the mapping function from real data and avoid an explicit model in terms of the PSF. A large amount of data is required for training the network that represents the mapping function. One way to obtain this data is to create blurred-unblurred image pairs by synthetically blurring images. However, these images do not capture effects such as real object and camera motion and occlusions. A system to obtain such data would in principle require a hybrid setup with 2 synchronized cameras, one with a long exposure and one with a short one. However, to avoid such setup altogether, we use a high frame rate video camera. The sharp image is taken as one of the frames and the blurry image as the average of all the frames. Fig. 1 shows an example of the ability of our network to generate sharp images despite space-variant blur.

Our contributions are: 1) A state-of-the-art blind deconvolution method for general space-varying motion blur (due to moving objects, depth variations and camera shake); 2) A novel convolutional network design motivated by the pyramid architecture; 3) The sharp image is directly obtained from our network very efficiently and without parameter tuning; 4) A novel database for sharp-motion blurred image pairs in the wild, which we will make available.

1.1. Related work

With the success of the variational Bayesian approach of Fergus et al. [9], a large number of blind deconvolution algorithms have been developed for motion deblurring [2, 5, 26, 37, 31, 28, 45, 42]. Although blind deconvolution algorithms consider blur to be uniform across the image, some of the methods are able to handle small variations due to camera shake [24]. Techniques based on blind deconvolution have been adapted to address blur variations due to camera rotations by defining the blur kernel on a higher dimensional space [11, 13, 40]. Another approach to handle camera shake induced space-variant blur is through region-wise blur kernel estimation [12, 19]. In 3D scenes, motion blur at a pixel is also related to depth at a point. To address this problem, techniques have been proposed that initially estimate a depth map and then solve for the motion blur PSF and the sharp image [16, 43]. In [46], motion blur due to forward or backward camera motion has been explicitly addressed. Note that all the above approaches consider blurring due to camera motion and are not suitable to handle sudden variations in blur due to moving objects.

Comparatively fewer methods exist for deblurring dynamic scenes. In an initial attempt to this problem by Levin [25], the blurred image is considered to be composed of different regions within which blur is assumed to be constant. Based on the effect of blur on image statistics, the segmentation map and blur estimates for different regions are initially derived. Subsequently, each region is separately deblurred through Richardson-Lucy deconvolution. Couzini et al. [6] perform deblurring for both space variant defocus and horizontal motion blur. They propose to estimate the space-variant PSF by combining learned local blur evidence with global smoothness constraints and then recover the sharp image using a deconvolution algorithm based on sparse regularization with global image statistics. The main restriction in these methods is that the blur model does not accurately represent the camera or object motion. Techniques based on alpha matting have been applied to restore scenes with two layers [7, 39]. Although these methods can handle moving objects, they require user interaction and cannot be used in general scenarios where blur varies due to camera motion and scene depth.

Kim et al. [20] proposed a segmentation-based technique to deblur dynamic scenes. The method incorporates alternate minimization to estimate blur kernels, latent image, and motion segments. Even with a general camera shake model for blurring, the algorithm fails in certain scenarios such as forward motion or depth variations [21]. In the technique proposed in [21] by Kim and Lee, abrupt changes in motion are handled without segmentation by a robust model. To reduce the complexity, motion blur is modeled by linear filters (uniform velocity assumption). The authors propose to simultaneously estimate motion flow and the latent image using a robust total variation (TV-L1) model. Schelten and Roth [32] propose a method to estimate and remove local non-parametric blur. Through a variational-Bayesian formulation, their framework recovers both defocus as well as object motion blur kernels. The sharp image is estimated by non-blind deblurring. Pan et al. [29] propose an efficient algorithm to jointly estimate object segmentation and camera motion by incorporating soft segmentation. In addition to the unknowns of sharp image and blur kernel of each layer, they introduce the terms of binary mask and a segmentation confidence map in their framework that allows handling of large blurs. Although the method produces good results in very challenging scenarios, the downside is that it requires user input for initializing the segmentation map and the number of segments. Another related approach to the deblurring problem is to use multiple frames. Recent works following this approach are able handle space-variant blur quite well [17, 41]. The works in [4, 10, 35] address the problem of segmenting an image into different regions according to blur. Although one could potentially estimate constant blur regions from these segmentation methods and apply standard deblurring techniques, in practice, such approaches would lead to artifacts at the boundaries unless handled suitably.

Since deep learning-based approaches have been able to achieve remarkable performance in different problems in computer vision, researchers have attempted to use them for motion deblurring. The methods in [33, 44] address the non-blind deconvolution problem wherein the sharp image is predicted using the PSF estimated from other techniques. In [34], Schuler et al. develop an end-to-end system that learns to perform blind deconvolution. Their system consists of modules to extract features, estimate the PSF and to perform deblurring. Similar to conventional blind deconvolution algorithm, they incorporate multi-scale iterations by repeating the set of modules. The performance of this approach degrades for larger sizes of blur kernels. Chakrabarti [3] proposes to learn the prediction of complex Fourier coefficients of a deconvolution filter for an input patch of the blurry image. From the restored patches, a global blur kernel and thence the sharp image are estimated through conventional optimization approaches. Hradiš et al. [14] predict clean and sharp images from text documents that are corrupted by motion blur, defocus and noise through a convolutional network without an explicit PSF estimation. This approach has been extended to license plates in [38]. While textual data contains repeated shapes and good data representation can be learnt for reconstruction, replicating this approach for natural images is significantly challenging. The deep learning-based methods discussed above are applicable for uniform blur scenarios. Sun et al. [36] propose to addresses non-uniform motion blur represented in terms of motion vectors with a specific length and orientation. They estimate probabilities of local blur kernels through a ConvNet and subsequently estimate dense motion kernels by enforcing motion smoothness. Finally space-variant deblurring is performed using patch-level image prior. Handling object motion could be difficult with this scheme because of the motion smoothness constraint.

2. Proposed formulation

Consider a dynamic scene imaged by a camera. The blurred observation g at a pixel $\mathbf{x}$ can be expressed as

$$g\left(\mathbf{x}\right) = \frac{1}{T_e}\int_0^{T_e} f_t\left(\mathbf{x}\right) dt \tag{1}$$

where f_t denotes the instantaneous image observed by the camera sensors at time t, and T_e denotes the exposure duration. One of the instantaneous (sharp) images within the exposure duration can be considered as the reference latent image f. If there are no occlusion effects, then f_t would be a transformed instance of the reference image f, *i.e.*, $f_t\left(\mathbf{x}\right) = f\left(M_t\left(\mathbf{x}\right)\right)$ for some mapping M_t. If the mapping is restricted to translations that are uniform across the image domain, then $M_t\left(\mathbf{x}\right) = \mathbf{x} - v_t$, with v_t a 2D vector that depends on time t, and Eqn. (1) can be replaced by the well-known convolution model

$$g\left(\mathbf{x}\right) = \frac{1}{T_e}\int_0^{T_e} f\left(\mathbf{x} - v_t\right) dt = \left(f * k\right)\left(\mathbf{x}\right) \tag{2}$$

where k denotes the shift-invariant PSF. However, in a general scenario, due to camera rotations, motion parallax, object motion and occlusion effects a simple and accurate model for Eqn. (1) seems elusive. Rather than enforcing a model on the data our approach is to discriminatively train a network using real data. We learn the parameters of a generative model that optimally estimates the sharp image given the blurred observation. To this purpose, we use a fully convolutional neural network that is trained in an end-to-end fashion: The network takes a blurry image as input and directly generates a sharp image estimate. Since deblurring is a demanding filtering task, we design a network with very high capacity (see Table 1). As a consequence, training such a bulky model becomes more demanding. We employ efficient techniques to get reasonable convergence of the network parameters, which we describe more in detail in the next section. Furthermore, this network requires a large set of blurry and sharp image pairs. To this end, we generate a new dataset using high resolution videos of dynamic scenes. The use of such a dataset is critical for achieving good performance as later seen in the experimental section. We describe the dataset generation more in detail in Sec. 3.

2.1. The Multiscale Convolutional Neural Network

Our convolutional neural network (CNN) architecture, shown in Fig. 2, has two main differences with respect to conventional CNNs. These differences play a significant role in generating high quality sharp images without artifacts. Firstly, it is designed in a pyramid or multi-scale fashion. Inspired by the multi-scale processing of blind deconvolution algorithms [28, 34], we include three segments N_1, N_2, and N_3 in our network, wherein each segment includes several convolution/deconvolution (fractional convolution) layers. The task of each segment is to minimize the reconstruction error at a particular scale. Because the extent of blur decreases with downsampling [28], the multi-scale formulation allows the network to deal with small amounts of blur in each segment. In particular, the task for the first segment N_1 is to generate a deblurred image at 1/4 of the original scale. The task for the segment N_2 is to start from this low resolution, but sharper, input and generate a sharp image at 1/2 of the original resolution. Finally, the task for the segment N_3 is to generate the sharp output at the original resolution. Secondly, the network is designed with a skip connection at the end of each segment. The idea behind this technique is to reduce the difficulty of the reconstruction task in the network by using the information already present in the blurry image. Thus, each segment needs to only generate a *residual image* which is then added to the

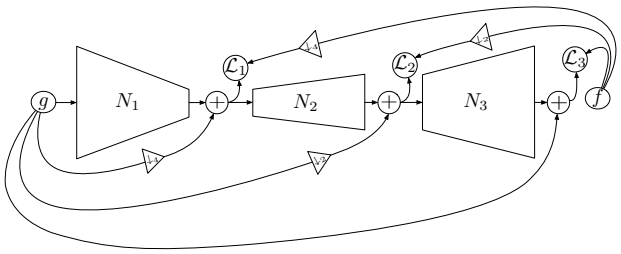


Figure 2: The Multiscale CNN architecture: The multiscale scheme allows the network to handle large blurs and skip connections facilitate the generation of details.

input blurry image (after downsampling, if needed). We observe experimentally that the skip connection technique helps the network in generating more texture details. We call this architecture the *Multiscale CNN*.

The first segment of our network generates sharp images at a scale corresponding to a downsampling factor of 4. This part consists of seven convolutional layers. In this segment, we perform downsampling by having stride of 2 in the first and fourth layers. The second and third segments include four convolutional layers followed by a deconvolutional layer with the stride of 2. The last deconvolution layer can be seen as a parametrized upsampling layer. The details of the architecture are shown in Table 1. During the training we minimize the reconstruction error of all the scales simultaneously. Our loss function $\mathcal{L}$ is defined as the sum of 3 losses:

$$\begin{aligned}
\mathcal{L} &= \mathcal{L}_1 + \mathcal{L}_2 + \mathcal{L}_3 \\
\mathcal{L}_1 &= \sum_{(g,f)\in\mathscr{D}} \left| N_1(g) + g^{\frac{1}{4}} - f^{\frac{1}{4}} \right|^2 \\
\mathcal{L}_2 &= \sum_{(g,f)\in\mathscr{D}} \left| N_2(N_1(g) + g^{\frac{1}{4}}) + g^{\frac{1}{2}} - f^{\frac{1}{2}} \right|^2 \\
\mathcal{L}_3 &= \sum_{(g,f)\in\mathscr{D}} \left| N_3(N_2(N1(g) + g^{\frac{1}{4}})) + g^{\frac{1}{2}}) + g - f \right|^2
\end{aligned} \tag{3}$$

where $\mathscr{D}$ is the training set, g denotes a blurry image, f denotes a sharp image, $I^{\frac{1}{k}}$ denotes the downsampling operation of the image I by factor of k, and N_i indicates the i-th segment in the Multiscale CNN, which reconstructs one scale.

2.2. Implementation Details

We used Adam [23] for optimization with momentum parameters as $\beta_1 = 0.9$, $\beta_2 = 0.999$, and an initial learning rate of 0.001. We decrease the learning rate by .75 every $10K$ iterations. We used two Titan X for training with a batch size of 10. The network needs 5 days to converge using batch normalization [18]. The filters of the first layer are shown in Fig 3. We observe that these filters are quite strong for an unsupervised learning method.

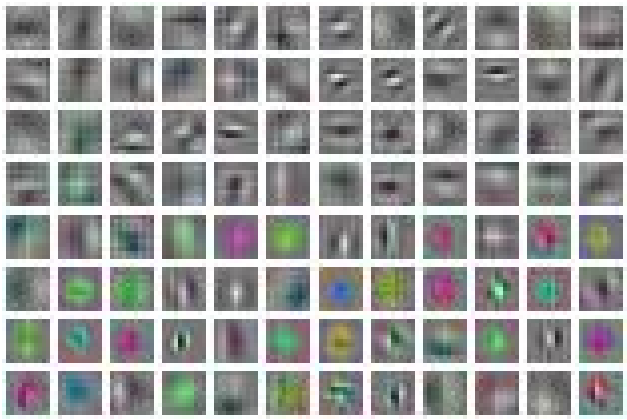


Figure 3: Filters of the first layer.

3. Blurry Images in the Wild

We train our network with a realistic dataset consisting of sharp and blurry observations of a dynamic scene obtained through a high resolution video camera. We average a set of frames to obtain realistic blurred observations. Similar averaging of frames has been done in previous works to obtain data for evaluation [22, 1]. While in [22] a blurry video is simulated, in [1] blurry images, coded exposure images and motion invariant photographs are synthesized.

From a handheld GoPro Hero5 camera which has a frame rate of 240 fps with a resolution of 1280×720, we collected a large set of videos of outdoor scenes. We downsampled all the frames in the videos by a factor of three in order to reduce the magnitude of relative motion across frames. The number of frames N_e to be averaged is selected at random from a set of odd numbers ranging between 7 and 23. Out of the N_e frames, the central frame is considered to be the sharp image. The averaging of frames corresponds to a discrete version of the integration in Eqn. (1). However, if the velocity of the relative motion is high, a scene point could jump more than 1 pixel across frames. Such jumps would not be occurring in a real motion blurred image. To avoid frames with higher displacement, we evaluate optical flow using the recent FlowNet algorithm [8]. We do not include those sets of frames in our dataset if the optical flow between adjacent frames is more than a pixel at any location. Fig. 4 shows an example of the sharp and blurred image pair in our training dataset. In this scene, we find both the camera and objects to be moving. By simply averaging the frames, we can embed the information corresponding to motion parallax, occlusions camera shake and

	N_1							N_2					N_3					
Type	conv	conv	conv	conv	conv	conv	conv	conv	conv	conv	conv	deconv	conv	conv	conv	conv	conv	deconv
OutCh	96	256	384	384	256	256	3	256	256	256	256	3	256	256	256	256	256	3
Kernel	11	7	7	7	3	3	3	5	5	5	5	5	5	5	5	5	5	5
Stride	$\downarrow 2$	1	1	$\downarrow 2$	1	1	1	1	1	1	1	$\uparrow 2$	1	1	1	1	1	$\uparrow 2$

Table 1: Details of the Multiscale CNN architecture. There is a Batch Normalization and Relu layer after every convolutional layer (except for the last layer of N_1) that are not shown for the sake of simplicity. Using a stride greater than one in convolutional layers downsamples the input and is indicated by the $\downarrow$ symbol. In contrast, a stride greater than one in deconvolutional layers performs upsampling and is indicated by the $\uparrow$ symbol.

object motion in our data.

For completeness, we also compute approximate PSFs in a sparse grid on the input image. The ideal PSF h at a location $\mathbf{x}$ can be related to the shifts undergone by the point during the exposure time as

$$h_{\mathbf{x}}(\mathbf{s}) = \frac{1}{T_e}\int_0^{T_e} \delta\left(\mathbf{s} - (\mathbf{x} - M_t(\mathbf{x}))\right) dt \qquad (4)$$

If we use the convolution relationship between f and g through the PSF h, we would arrive at the imaging model in Eqn. 1. In the case of our dataset, we arrive at an approximate estimate of the PSF using the optical flow values returned by the algorithm in [8]. We show the approximate estimates of the PSF in Fig. 4 (c).

4. Experiments

We tested the performance of our method on three different types of data: a) a test set in the wild created from a high frame rate video camera, b) real images captured from a Canon EOS 5D, and c) data from previous works. Additional results and comparisons are presented in the supplementary material.

To verify the impact of using a realistic dataset for space-variant blurring, we trained another network with the same architecture as the one in Fig. 2. However, we used synthetically blurred images according to the convolution model. As in [3], we prepared a set of $100K$ different PSFs. During training, we randomly pick a motion PSF and convolve it with a sharp texture to generate blurred data. We refer to this network as the *MSCNN(SI)*, multiscale CNN space-invariant (data), and include comparisons of this network with our method. A second network is instead trained only on the blurry-sharp image pairs from our dataset in the wild. This network is called *MSCNN(WILD)*, multiscale CNN from data in the wild.

4.1. Test-set evaluation

The videos in the test set were captured at locations different from those where training data was captured. Also incidentally, the weather conditions were significantly different when the test set was captured. We randomly chose 15 images from the test-set and compared the performance of our method against the methods in [42], [36], the space variant implementation of the method in [45], and our MSCNN(SI) and MSCNN(WILD) methods. Representative examples corresponding to two different scenes are shown in Fig. 5. Although we observe deblurring from the methods in [45] and [42] at a few locations, these methods fail when there is blur variation either due to object motion or depth changes. Our results are closer to the true image. Using the reference images, we quantified the performance of the methods in terms of PSNR as shown in Table 2. The PSNR values indicate that both our methods perform better than previous work. However, the MSCNN(WILD) method's performance is *significantly better* than MSCNN(SI). Since the test data set is quite challenging due to factors such as reflections/specularities and space variant blur, the PSNR values are not comparable to the values seen in conventional deconvolution algorithms. We show all the 15 images from the test set and comparisons with other methods in the supplementary material.

[36]	[42]	[45]	MSCNN(SI)	MSCNN(WILD)
25.48	23.61	22.50	25.8	**28.1**

Table 2: Average PSNR on our test set in the wild.

4.2. Qualitative evaluation

We also evaluate our network qualitatively on datasets of prior works and images obtained from a Canon camera. While Figs. 6 and 7, consist of data from [36] and [21], respectively, Fig. 8 shows images from our camera. In Fig. 8, we compare the methods of [42], [36] and [45] to our MSCNN(SI) and MSCNN(WILD). In all the images, we can see that our method is able to return sharper images with more details. Further, we observe that in Fig. 8, MSCNN(WILD) produces significantly better results than MSCNN(SI).



(a) (b) (c)

Figure 4: Example of training data: (a) Averaged image. (b) Reference frame. (c) PSF estimated from optical flow.

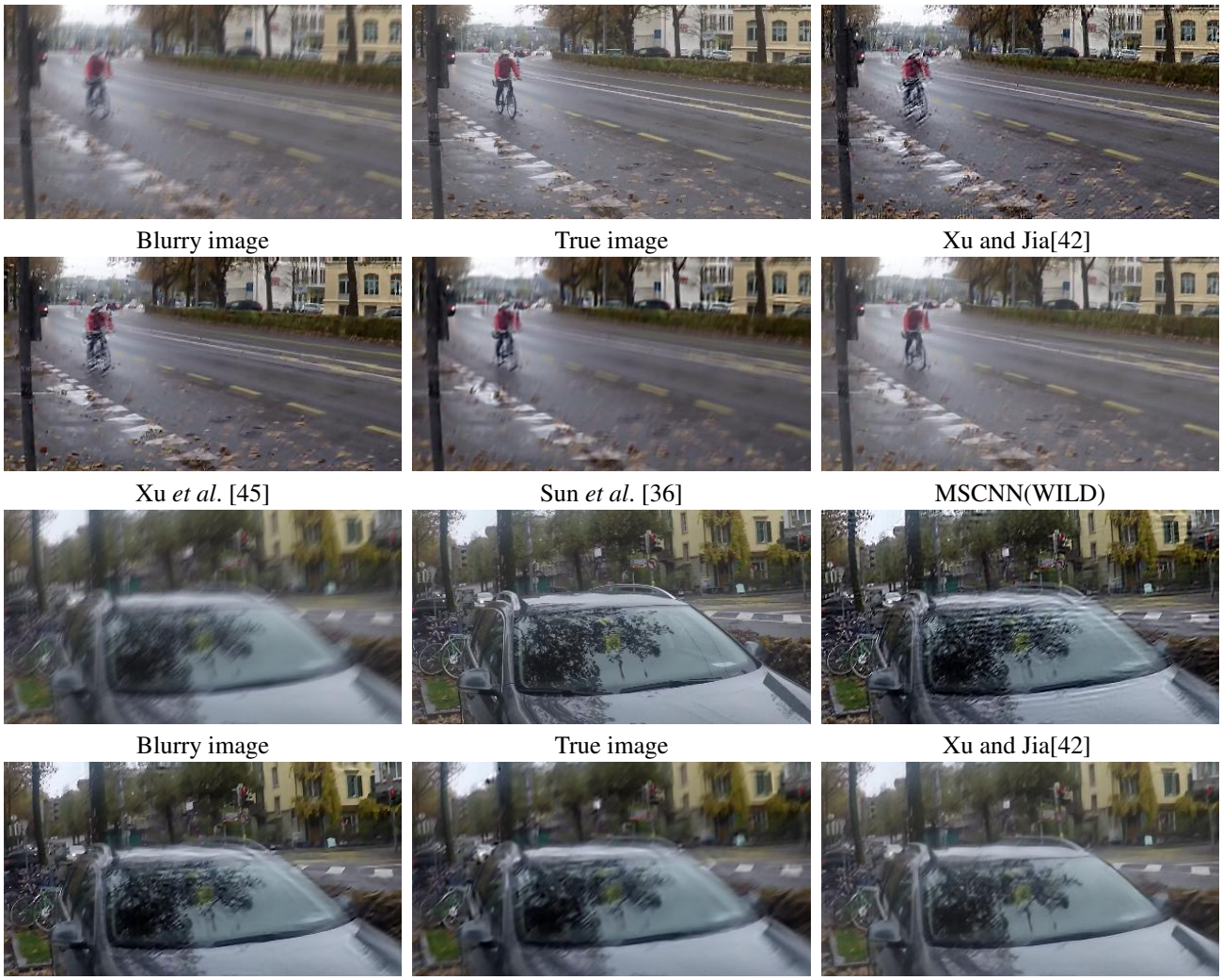
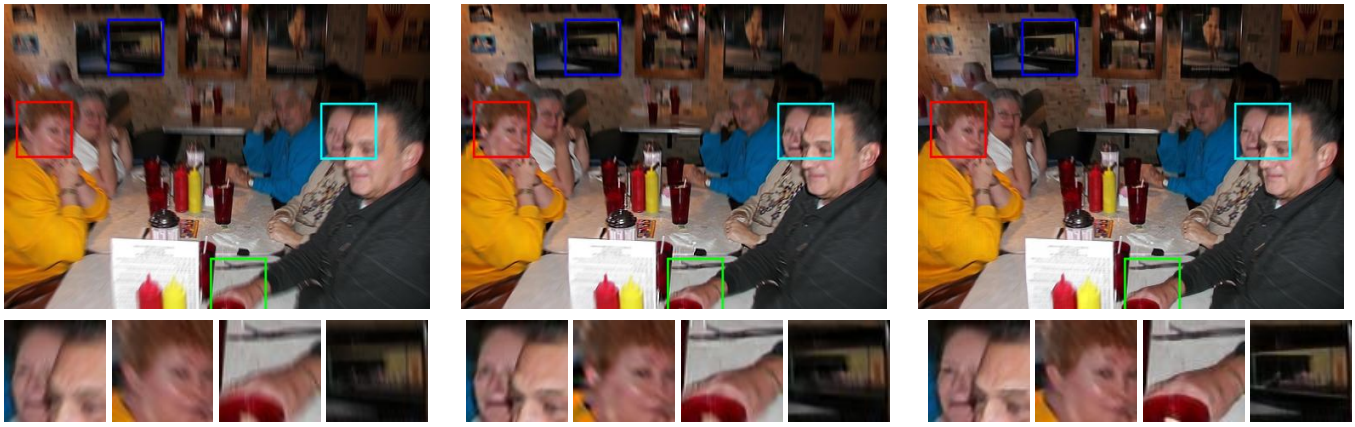


Figure 5: Evaluation on our test dataset in the wild.

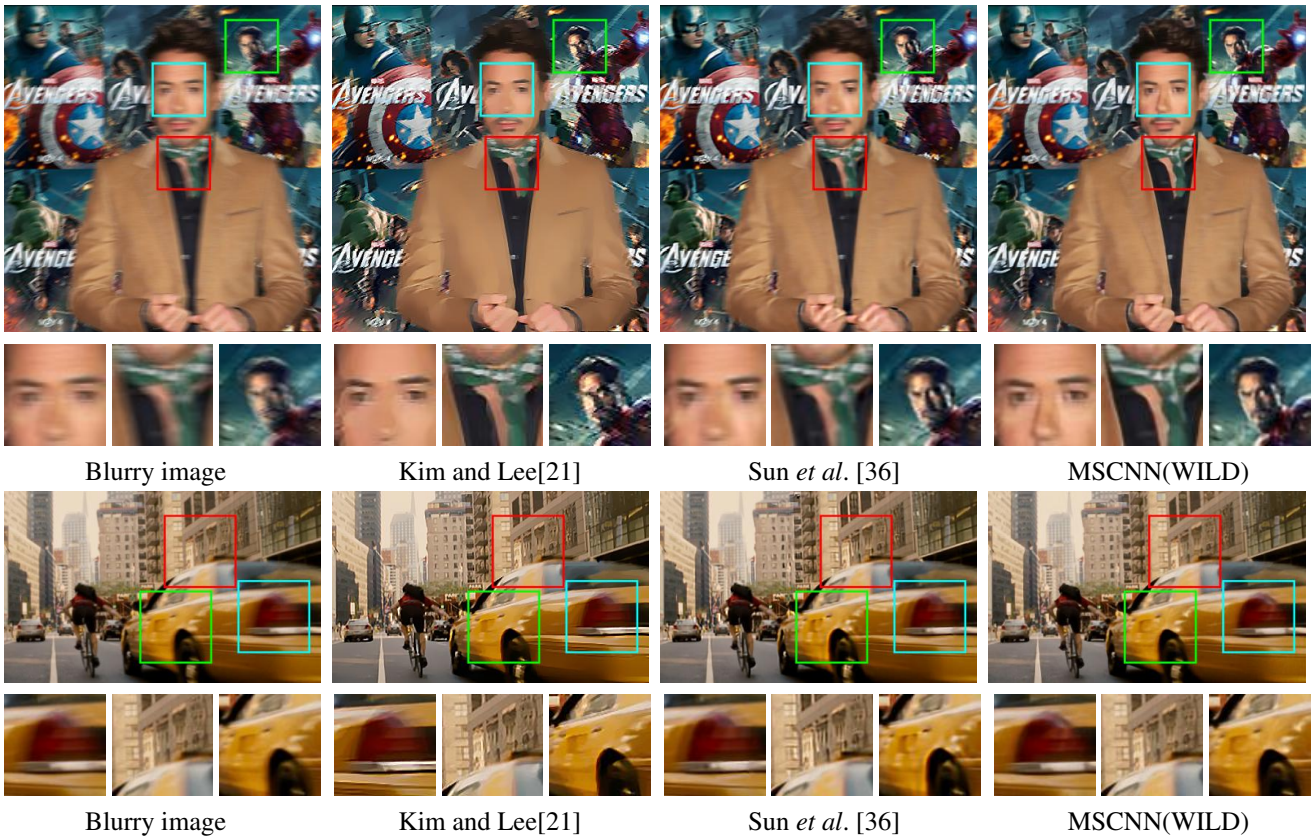
5. Conclusions

We proposed a novel convolutional neural network architecture for predicting a sharp image given a blurred one.



Blurry image | Sun *et al*. [36] | MSCNN(WILD)

Figure 6: Evaluation on the test dataset from [36].



Blurry image | Kim and Lee[21] | Sun *et al*. [36] | MSCNN(WILD)

Blurry image | Kim and Lee[21] | Sun *et al*. [36] | MSCNN(WILD)

Figure 7: Evaluation on the test dataset [21].

Our network is able to infer estimates of a sharp image under challenging scenarios due to occlusions, saturation, motion parallax and camera rotations. The network is designed in 3 segments and it exploits a multiscale strategy to break down the complexity of the deblurring task. Moreover, each segment outputs only a residual image that yields the sharp image when added to the input image. This allows the segment to focus on small details as confirmed experimentally.

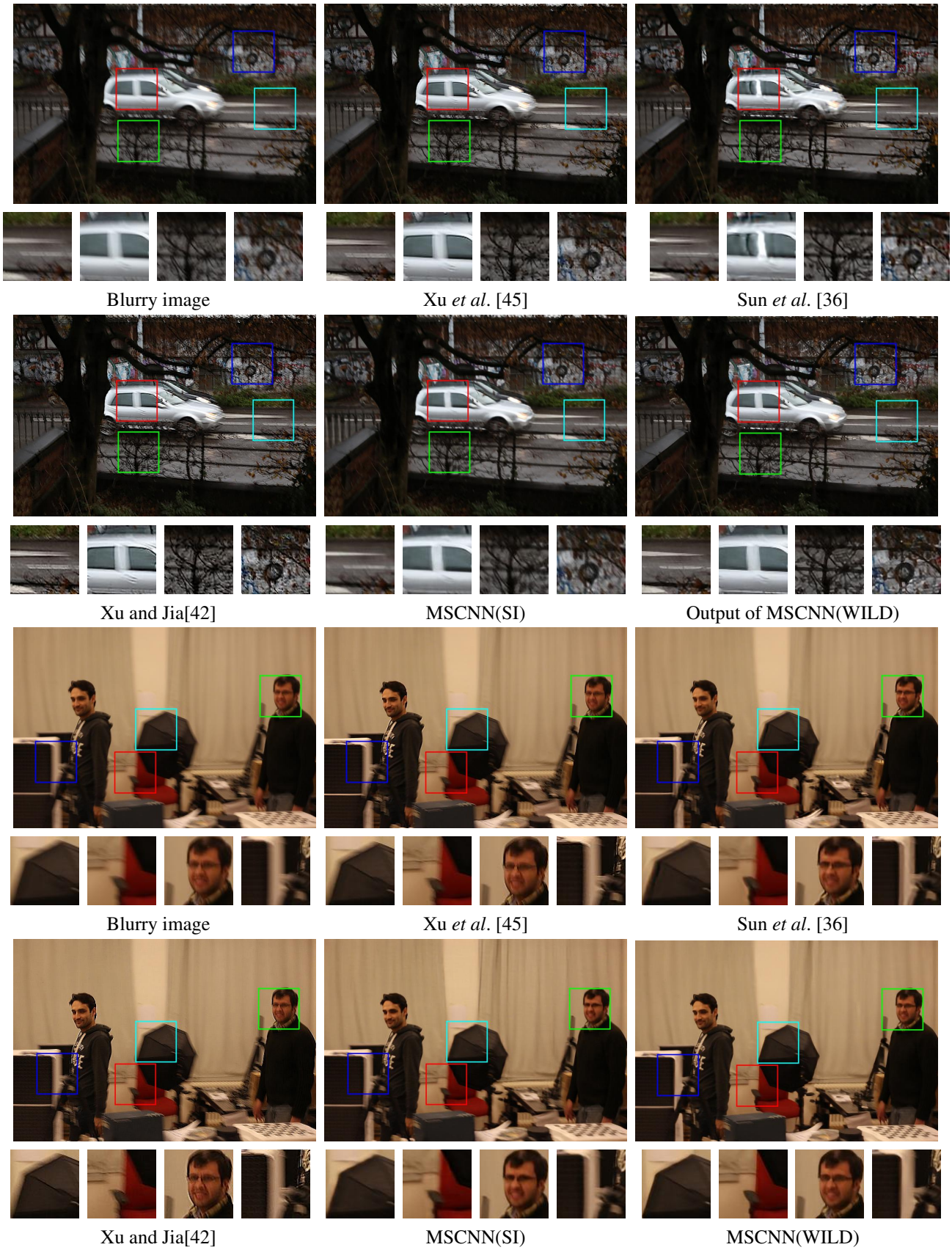


Figure 8: Evaluation on images from the Canon camera.

An important part of our solution is the design of a realistic dataset. We find that simple frame averaging combined with a very high frame rate video camera produces reasonable blurred-sharp image pairs for the training of our network. Indeed, both quantitative and qualitative results show a good performance when compared to prior work. Moreover, we observe that our network does not generate artifacts, but may leave extreme blurs untouched. In future work we plan to use the same strategy to address more challenging scenarios due to saturations and reflections.

References

[1] A. Agrawal and R. Raskar. Optimal single image capture for motion deblurring. In *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*, pages 2560–2567. IEEE, 2009.

[2] S. D. Babacan, R. Molina, M. N. Do, and A. K. Katsaggelos. Bayesian blind deconvolution with general sparse image priors. In *ECCV*, Firenze, Italy, October 2012. Springer.

[3] A. Chakrabarti. A neural approach to blind motion deblurring. In *ECCV*, 2016.

[4] A. Chakrabarti, T. Zickler, and W. T. Freeman. Analyzing spatially-varying blur. In *Computer Vision and Pattern Recognition (CVPR), 2010 IEEE Conference on*, pages 2512–2519. IEEE, 2010.

[5] S. Cho and S. Lee. Fast motion deblurring. *ACM Trans. Graph.*, 28(5):1–8, 2009.

[6] F. Couzinie-Devy, J. Sun, K. Alahari, and J. Ponce. Learning to estimate and remove non-uniform image blur. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1075–1082, 2013.

[7] S. Dai and Y. Wu. Removing partial blur in a single image. In *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*, pages 2544–2551. IEEE, 2009.

[8] A. Dosovitskiy, P. Fischer, E. Ilg, P. Hausser, C. Hazirbas, V. Golkov, P. van der Smagt, D. Cremers, and T. Brox. Flownet: Learning optical flow with convolutional networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2758–2766, 2015.

[9] R. Fergus, B. Singh, A. Hertzmann, S. T. Roweis, and W. T. Freeman. Removing camera shake from a single photograph. *ACM Trans. Graph.*, 25(3):787–794, 2006.

[10] J. Gast, A. Sellent, and S. Roth. Parametric object motion from blur. *arXiv preprint arXiv:1604.05933*, 2016.

[11] A. Gupta, N. Joshi, L. Zitnick, M. Cohen, and B. Curless. Single image deblurring using motion density functions. In *ECCV*, 2010.

[12] M. Hirsch, C. J. Schuler, S. Harmeling, and B. Schölkopf. Fast removal of non-uniform camera shake. In *2011 International Conference on Computer Vision*, pages 463–470. IEEE, 2011.

[13] M. Hirsch, S. Sra, B. Schölkopf, and S. Harmeling. Efficient filter flow for space-variant multiframe blind deconvolution. In *CVPR*, 2010.

[14] M. Hradiš, J. Kotera, P. Zemcík, and F. Šroubek. Convolutional neural networks for direct text deblurring. In *BMVC*, 2015.

[15] Z. Hu, L. Xu, and M.-H. Yang. Joint depth estimation and camera shake removal from single blurry image. In *CVPR*, pages 2893–2900. IEEE, 2014.

[16] Z. Hu, L. Xu, and M.-H. Yang. Joint depth estimation and camera shake removal from single blurry image. In *CVPR*, 2014.

[17] T. Hyun Kim and K. Mu Lee. Generalized video deblurring for dynamic scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5426–5434, 2015.

[18] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.

[19] H. Ji and K. Wang. A two-stage approach to blind spatially-varying motion deblurring. In *Computer Vision and Pattern Recognition (CVPR), 2012 IEEE Conference on*, pages 73–80. IEEE, 2012.

[20] T. H. Kim, B. Ahn, and K. M. Lee. Dynamic scene deblurring. In *The IEEE International Conference on Computer Vision (ICCV)*, December 2013.

[21] T. H. Kim and K. M. Lee. Segmentation-free dynamic scene deblurring. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 2766–2773. IEEE, 2014.

[22] T. H. Kim, S. Nah, and K. M. Lee. Dynamic scene deblurring using a locally adaptive linear blur model. *arXiv preprint arXiv:1603.04265*, 2016.

[23] D. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[24] R. Köhler, M. Hirsch, B. J. Mohler, B. Schölkopf, and S. Harmeling. Recording and playback of camera shake: Benchmarking blind deconvolution with a real-world database. In *ECCV (7)*, pages 27–40, 2012.

[25] A. Levin. Blind motion deblurring using image statistics. In *NIPS*, 2006.

[26] A. Levin, Y. Weiss, F. Durand, and W. Freeman. Efficient marginal likelihood optimization in blind deconvolution. In *CVPR*, pages 2657–2664, 2011.

[27] A. Levin, Y. Weiss, F. Durand, and W. T. Freeman. Understanding and evaluating blind deconvolution algorithms. In *CVPR*, pages 1964–1971. IEEE, June 2009.

[28] T. Michaeli and M. Irani. Blind deblurring using internal patch recurrence. In *ECCV*, pages 783–798. Springer, 2014.

[29] J. Pan, Z. Hu, Z. Su, H.-Y. Lee, and M.-H. Yang. Soft-segmentation guided object motion deblurring. In *Proceedings of the IEEE International Conference on Computer Vision*, 2016.

[30] C. Paramanand and A. N. Rajagopalan. Non-uniform motion deblurring for bilayer scenes. In *CVPR*, 2013.

[31] D. Perrone and P. Favaro. Total variation blind deconvolution: The devil is in the details. In *CVPR*, 2014.

[32] K. Schelten and S. Roth. Localized image blur removal through non-parametric kernel estimation. In *International Conference on Pattern Recognition*, 2014.

[33] C. J. Schuler, H. Christopher Burger, S. Harmeling, and B. Scholkopf. A machine learning approach for non-blind image deconvolution. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2013.

[34] C. J. Schuler, M. Hirsch, S. Harmeling, and B. Schölkopf. Learning to deblur. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 38(7):1439–1451, 2016.

[35] J. Shi, L. Xu, and J. Jia. Discriminative blur detection features. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2965–2972, 2014.

[36] J. Sun, W. Cao, Z. Xu, and J. Ponce. Learning a convolutional neural network for non-uniform motion blur removal. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 769–777, 2015.

[37] L. Sun, S. Cho, J. Wang, and J. Hays. Edge-based blur kernel estimation using patch priors. In *ICCP*, pages 1–8. IEEE, 2013.

[38] P. Svoboda, M. Hradiš, L. Maršík, and P. Zemcík. Cnn for license plate motion deblurring. In *ICIP*, 2016.

[39] Y.-W. Tai, N. Kong, S. Lin, and S. Y. Shin. Coded exposure imaging for projective motion deblurring. In *Computer Vision and Pattern Recognition (CVPR), 2010 IEEE Conference on*, pages 2408–2415. IEEE, 2010.

[40] O. Whyte, J. Sivic, A. Zisserman, and J. Ponce. Non-uniform deblurring for shaken images. In *CVPR*, 2010.

[41] P. Wieschollek, B. Schölkopf, H. P. A. Lensch, and M. Hirsch. End-to-end learning for image burst deblurring. In *ACCV*, 2016.

[42] L. Xu and J. Jia. Two-phase kernel estimation for robust motion deblurring. In *ECCV*, 2010.

[43] L. Xu and J. Jia. Depth-aware motion deblurring. In *ICCP*, pages 1 –8, april 2012.

[44] L. Xu, J. S. Ren, C. Liu, and J. Jia. Deep convolutional neural network for image deconvolution. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27*, pages 1790–1798. Curran Associates, Inc., 2014.

[45] L. Xu, S. Zheng, and J. Jia. Unnatural l0 sparse representation for natural image deblurring. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1107–1114, 2013.

[46] S. Zheng, L. Xu, and J. Jia. Forward motion deblurring. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1465–1472, 2013.

Supplementary Material

This supplementary material includes two sections. In the first section we provide more results and comparisons. In the second section we provide some analysis on our network.

6. Additional results

In the main paper we showed the average PSNRs of the methods by Xu and Jia [42], Xu et al. [45], and Sun et al.[36] on the 15 images of our test set. In Figs. 10-14 we show the visual outputs of all methods including ours and report the corresponding PSNR in the caption. Furthermore, the images with the highest (lowest) PSNR for each method are depicted by a green (red) box. Since the code of [21] is not publicly available, we could not show their results.

In Figs. 15, 16 we also show a visual comparison of all the methods above when run on images from other data sets. In this case we also show the results of Kim and Lee [21]. In Fig.17 we could also show results of [21] on two Canon images, which were kindly provided by the authors shortly after the paper submission deadline.

Notice that our method behaves conservatively: It removes blur without introducing artifacts or it leaves the original content (*e.g.*, when blur is too large). Other methods may instead oversharpen or introduce oscillations that spoil the image content.

The first image in Fig. 15 is captured with a Canon EOS 5D Mark II camera. This blurry image is obtained directly from the camera without averaging or any other post-processing. Notice that this is a very challenging scenario since there is both camera shake and objects are moving independently. We also show details of the outputs in insets below each image. Our method can produce sharper results on diverse regions such as faces, heads, necks, clothing, than other competing algorithms. The method of Kim and Lee [21] can deblur successfully some regions (*e.g.*, neck) but also introduces artifacts to the back of the head. In contrast, our method can deal with substantial blurs, such as that on the hand, and reduce them without corrupting the image. The second image (canal) is taken from [46]. Similar considerations can be drawn also in this case.

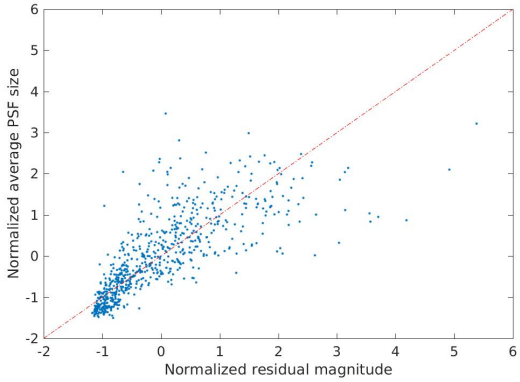


Figure 9: Analysis. Normalized average PSF size versus normalized residual magnitude plot. Notice the high level of correlation between the blur size and the residual magnitude.

In Fig. 16 we show results on a data set from [21]. As shown in the insets below each image, our method clearly outperforms the others. While all methods perform some deblurring, our method can debur more textures and regions. In the car image, [42] obtains image sharpness at the cost of adding artifacts. [21] yields an overly-cartooned image and thus loses details (*e.g.*, see the back of the van).

7. Analysis

Because our network is very deep we analyze only the very last layer and leave the analysis of the other layers to future work. Our network generates a residual image that when added to the blurry input yields the sharp image. The blurrier the image, the larger residual, as more changes will be required. Thus, in the ideal case, blur is related to the magnitude of the residual, which we also call *activation*. We therefore expect a good correlation between our generated residual and the blur magnitude of the input blurry image.

Our experiments validate this hypothesis quantitatively and qualitatively. We take 700 images of the test set of our dataset, feed them to the net-

work, and calculate their activations, *i.e.*, the l_1 norm of the network outputs (the residuals). In Fig. 18 we show 6 most activating images, *i.e.*, images with the highest l_1 norm in the output, and in Fig. 19 we show 6 least activating images, *i.e.*, images with lowest l_1 norm in the output. In these images each of the three columns contains two examples. The image in the first row is the input, the second row shows the estimated residual (the network output), the third row is the deblurred image (first row + second row), and finally the forth row is the ground truth. As expected, the most activating images are highly blurred, while the least activating ones are sharp samples.

We also show quantitatively that there is a nice correlation between blurriness and the residual magnitude. As we have mentioned in the main paper, our dataset stores an estimate of point spread functions (PSFs) obtained from optical flow for each blurry image. We calculate the average PSF size across the blurry image, and this gives us a reasonable approximation of blurriness (the approximation is due to the error in the optical flow estimates). In Fig. 9 we show the plot of the l_1 norm of the residual versus average estimated PSF size from optical flow for all 700 images. The residual magnitudes and blur sizes are normalized so that mean and std of both are 0 and 1 respectively. Notice the high level of correlation between the blur size and the activation.



Blurry image(28.5) | Ground truth | Xu and Jia [42] (25.3)



Xu *et al*. [45] (22.3) | Sun *et al*. [36] (29.1) | Ours (31.1)



Blurry image(27.9) | Ground truth | Xu and Jia [42] (23.3)



Xu *et al*. [45] (24.2) | Sun *et al*. [36] (27.3) | Ours (30.5)



Blurry image(28.0) | Ground truth | Xu and Jia [42] (23.3)



Xu *et al*. [45] (22.7) | Sun *et al*. [36] (27.2) | Ours (30.8)

Figure 10: Quantitative evaluation. Results of all 15 images of our test dataset in the wild. The PSNR of all methods and for each image is shown next to the authors name in parentheses. Images with highest/ lowest PSNR are highlighted with a green/ red box.



Figure 11: Quantitative evaluation. Results of all 15 images of our test dataset in the wild. The PSNR of all methods and for each image is shown next to the authors name in parentheses. Images with highest/ lowest PSNR are highlighted with a green/ red box.



Figure 12: Quantitative evaluation. Results of all 15 images of our test dataset in the wild. The PSNR of all methods and for each image is shown next to the authors name in parentheses. Images with highest/ lowest PSNR are highlighted with a green/ red box.

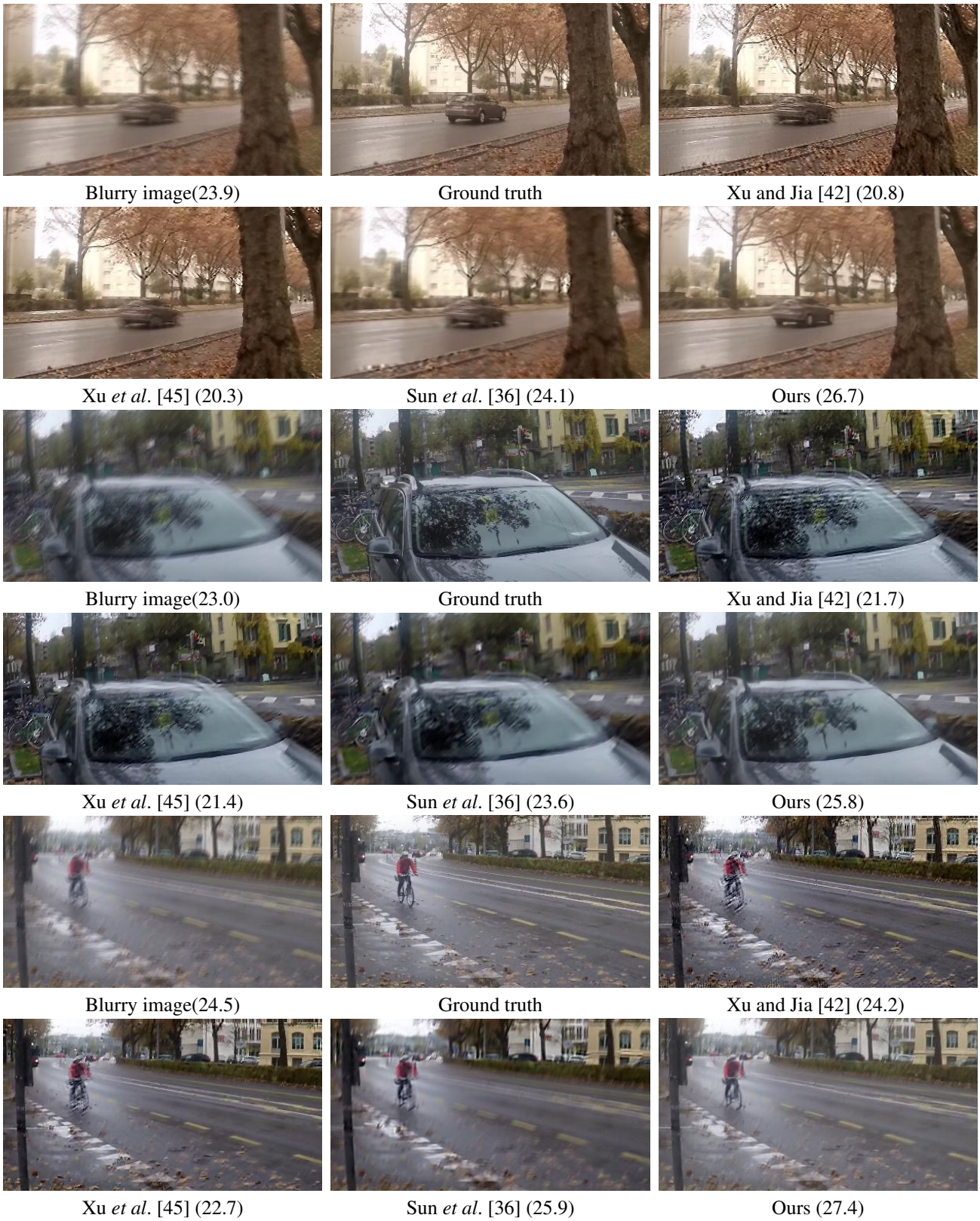


Figure 13: Quantitative evaluation. Results of all 15 images of our test dataset in the wild. The PSNR of all methods and for each image is shown next to the authors name in parentheses. Images with highest/ lowest PSNR are highlighted with a green/ red box.

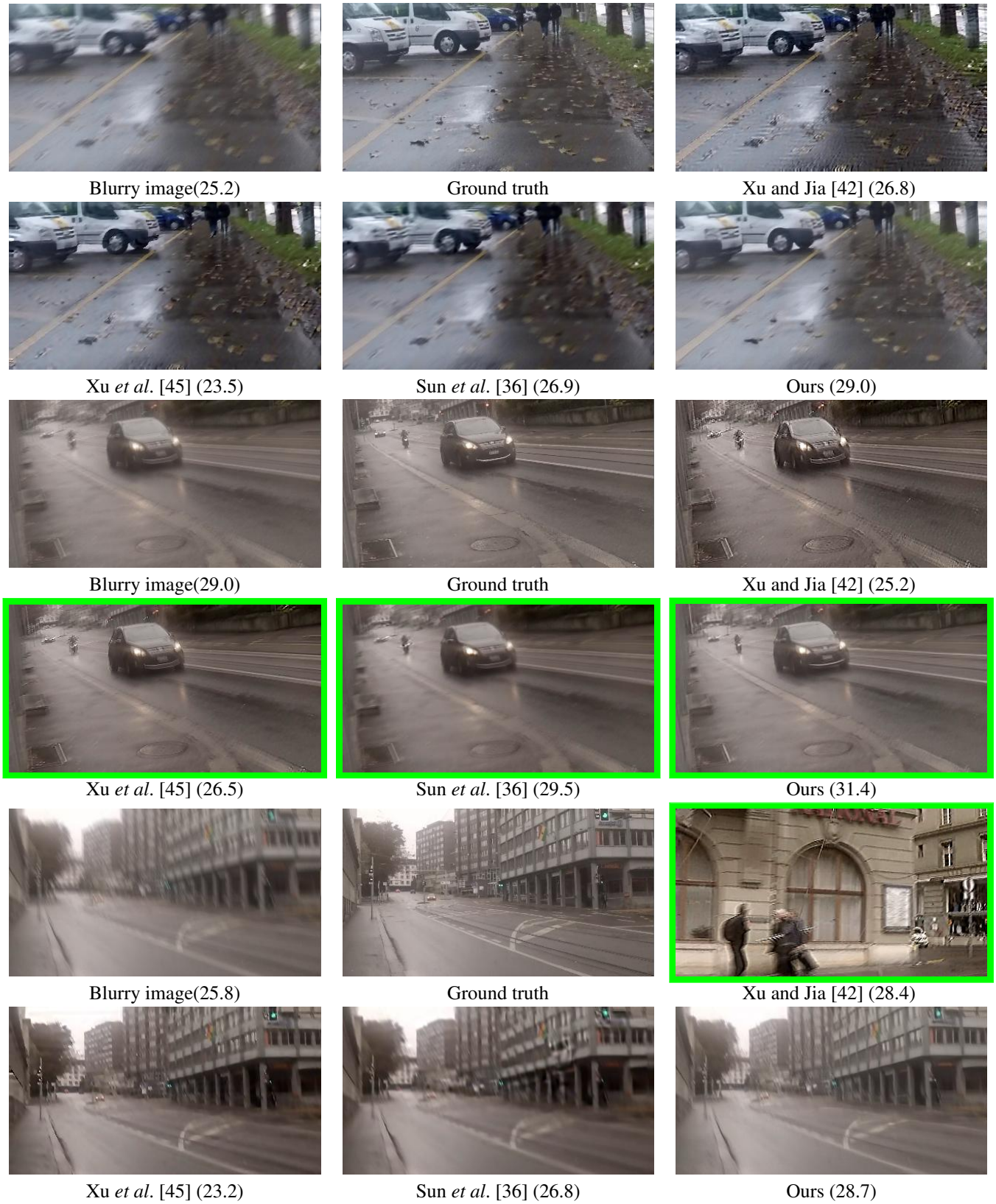


Figure 14: Quantitative evaluation. Results of all 15 images of our test dataset in the wild. The PSNR of all methods and for each image is shown next to the authors name in parentheses. Images with highest/ lowest PSNR are highlighted with a green/ red box.

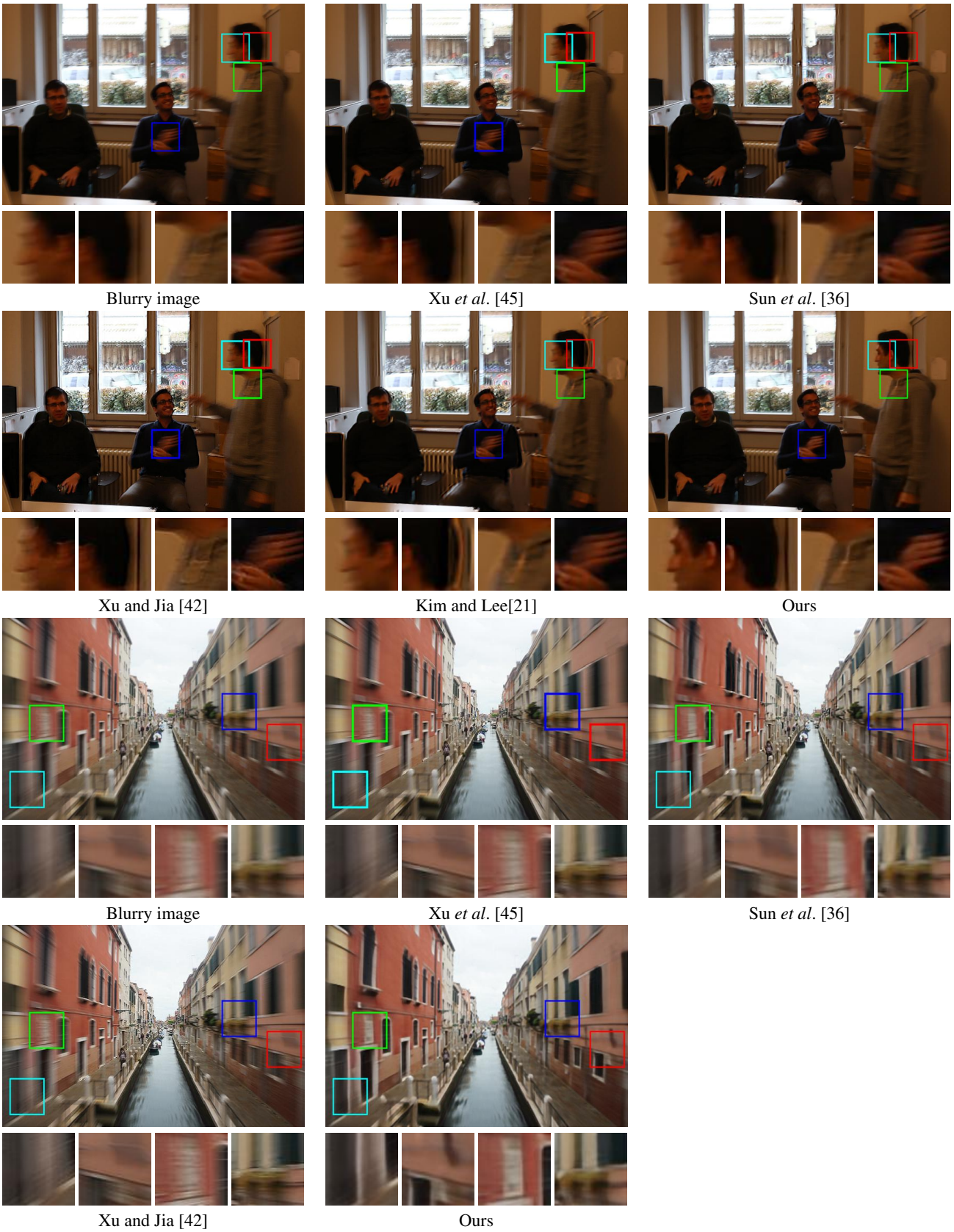


Figure 15: Qualitative evaluation. Additional results from external datasets where ground truth is not available. The image on the top is from our Canon EOS 5D Mark II dataset. The bottom image is taken from [46].



Figure 16: Qualitative evaluation. Additional results from external datasets where ground truth is not available. Both images are taken from Kim and Lee [21].

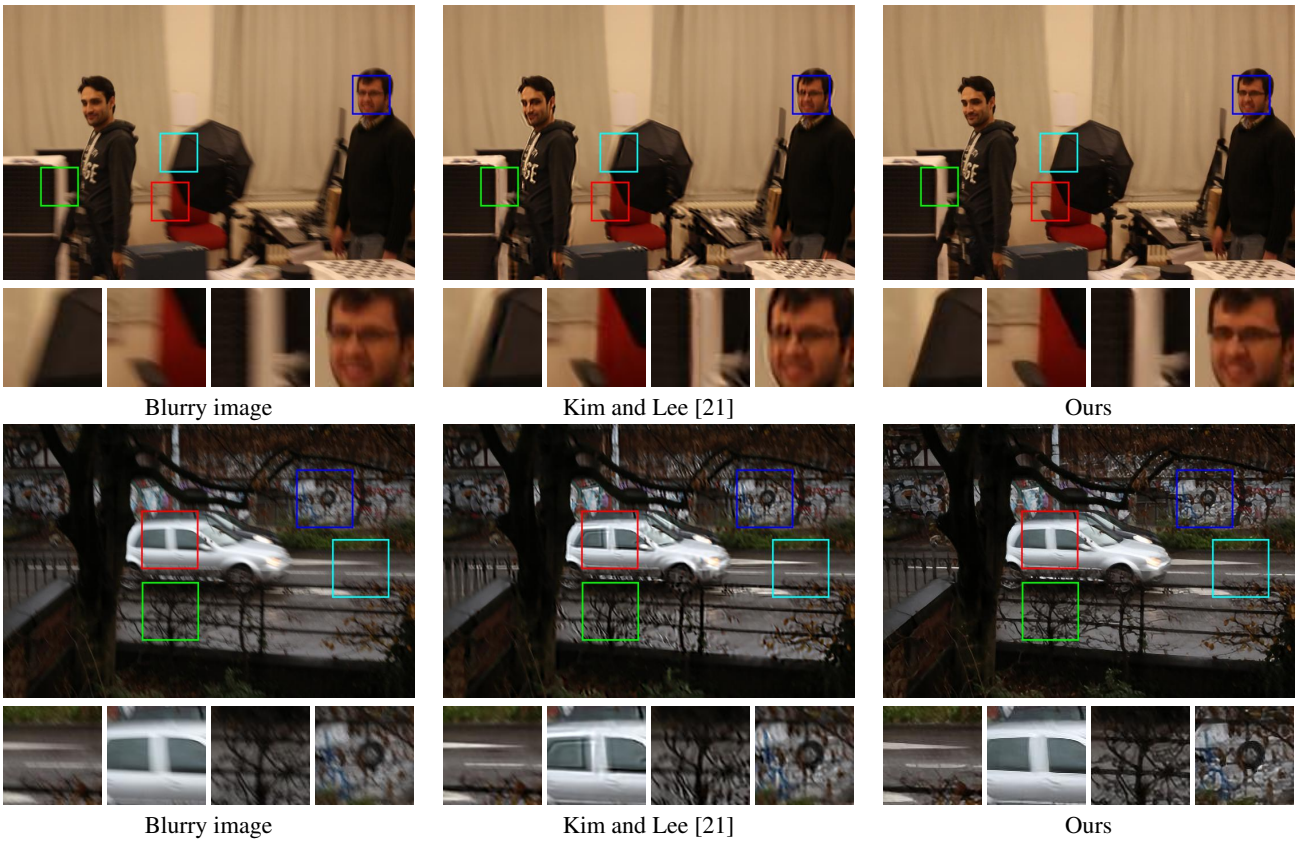


Figure 17: Qualitative comparison with Kim and Lee [21] on two Canon EOS 5D Mark II images. We compare the results of more methods on the same images in the main paper. We repeat these images here since the results of Kim and Lee [21] were kindly provided by the authors shortly after the paper submission deadline.

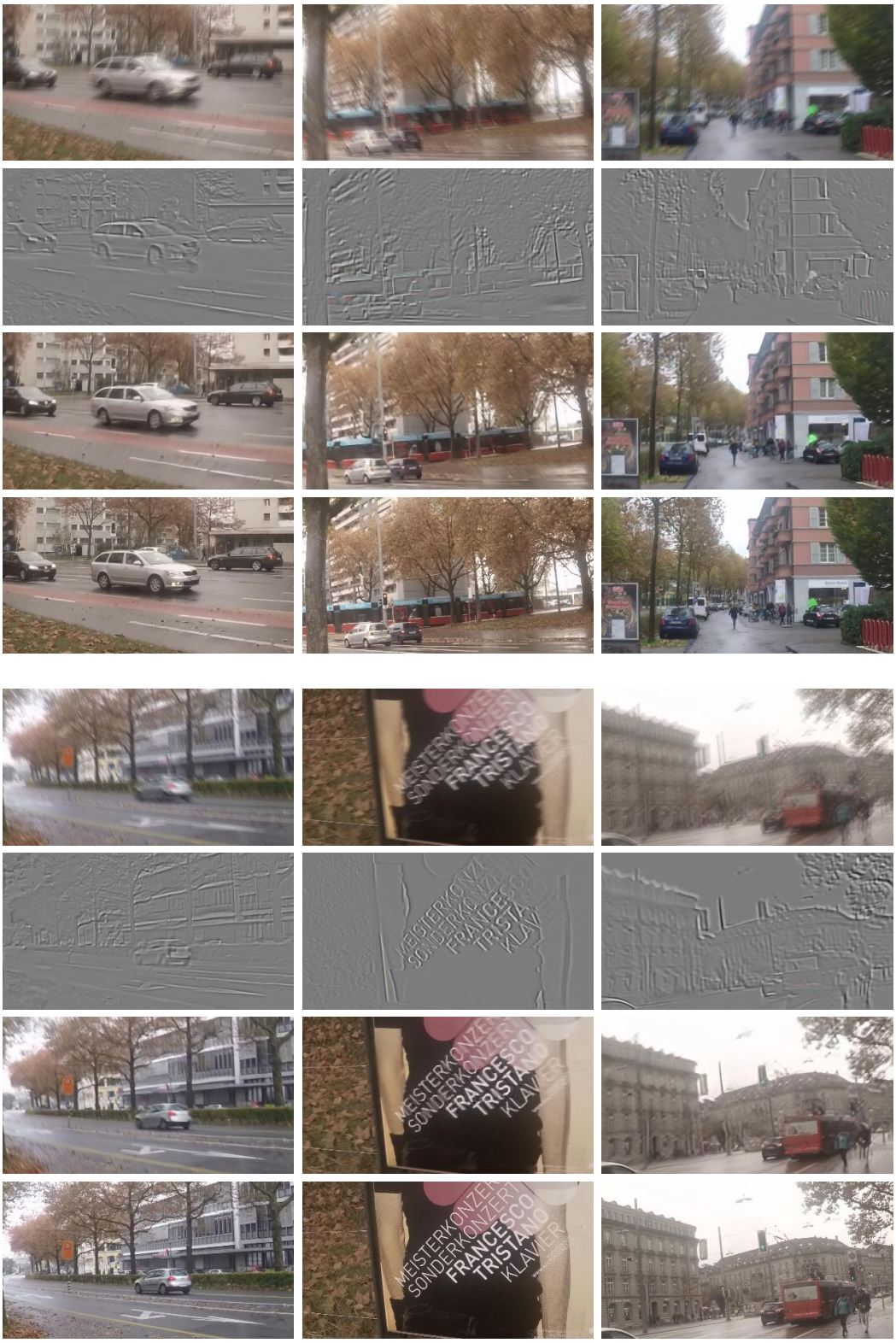


Figure 18: Analysis. The images with highest activations in the output layer. In these images each of the three columns contains two examples. The image in the first row is the input, the second row shows the estimated residual (the network output), the third row is the deblurred image (first row + second row), and finally the forth row is the ground truth.

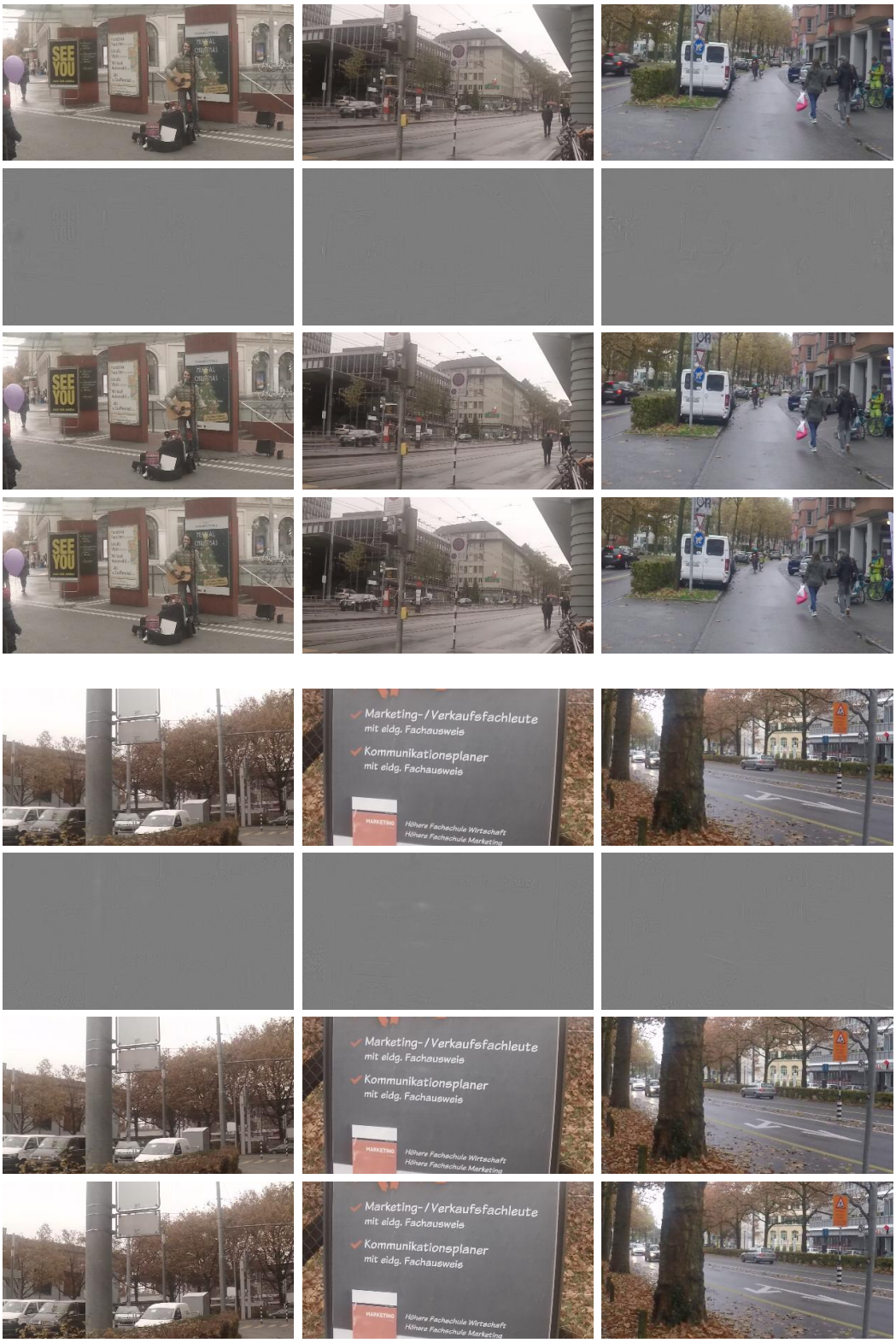


Figure 19: Analysis. The images with lowest activations in the output layer. In these images each of the three columns contains two examples. The image in the first row is the input, the second row shows the estimated residual (the network output), the third row is the deblurred image (first row + second row), and finally the forth row is the ground truth.